

SOFA: Service-Oriented Fine-Grained Attack Traffic Detection With Meta Learning

Peng Fang[✉], Feiyang Huang, Ziming Zhao[✉], *Member, IEEE*, Fan Zhang[✉], *Member, IEEE*, and Wenrui Ma[✉]

Abstract—Enterprise networks face an ever-growing threat from various unknown cyberattacks originating from the Internet. Anomaly-based Network Intrusion Detection Systems (NIDS) have become essential for safeguarding corporate networks, but existing technologies have some limitations in practice. (i) The interweaving of traffic from different service types increases the difficulty of identification. (ii) The scarcity of labeled malicious samples hinders the detection of both known and emerging threats. To tackle these issues, we propose a service-oriented approach to model benign traffic and leverage meta-learning to construct a robust metric space for precise sample comparison. Based on this, we develop SOFA, a two-stage traffic detection framework. In the first stage, SOFA identifies distinct service types within the network and trains independent one-class models for each, ensuring service-specific attack traffic is detected with high precision. The second stage employs a ResNet-based Siamese network to overcome the limitations posed by scarce malicious samples, enhancing detection of both known and emerging attacks. We evaluate SOFA on four widely used network tracing datasets, demonstrating that it achieves state-of-the-art performance and significantly outperforms a diverse set of existing methods in fine-grained attack detection. These results highlight the potential of SOFA for improving network security in a highly dynamic and unbalanced threat environment.

Index Terms—Fine-grained attack detection, unknown attack detection, Siamese network, service-oriented scheme.

I. INTRODUCTION

ENTERPRISE networks are increasingly targeted by sophisticated cyberattacks, including both known and unknown threats [1], [2]. Protecting these networks requires advanced and scalable mechanisms to detect and mitigate such threats. Among existing approaches, Network Intrusion Detection Systems (NIDS) have undergone significant advancements [3], evolving from signature-based [4], [5] to anomaly-based detection methods [6], [7], [8], driven by the rise of encryption technologies and the growing prevalence of unknown threats. Although the anomaly-based intrusion detection scheme has

achieved certain effectiveness, existing methods still have some limitations in practice [9]. We summarize the two challenges as follows.

(i) *The interweaving of traffic from different service types increases the difficulty of identification.* In existing traffic classification studies, previous schemes fed all benign traffic types (e.g., SSH, Web, FTP) into the model en bloc [10], [11], [12]. However, benign traffic comprises various applications with distinct behavioral patterns. This leads to misclassifications where a malicious session (a session refers to a bidirectional flow, uniquely identified by a 5-tuple $\langle srcip, srcport, dstip, dstport, protocol \rangle$) may be closer to an unrelated benign session than to other malicious ones [13]. For better understanding, we trained a one-class learner [14] with benign traffic on a specific port (e.g., TCP port 21, 22 and 80) and tested it on various malicious traffic. The results in Fig. 1 uncover notable misclassification instances. For example, DDoS attacks are hard to distinguish from normal SSH traffic (Fig. 1a). Botnet activity on port 8080 overlaps with benign web traffic on port 80 (Fig. 1b) and FTP services on port 21 (Fig. 1c). However, these misclassification instances occur on traffic of different types of services. Therefore, if we use service type to perform a preliminary screening on traffic, a one-class learner will not struggle with these unnecessary misclassifications.

Some studies analyze benign traffic by category (e.g., videos, emails) [14] or similarity (e.g., K-means clustering) [15]. However, these methods fail to resolve the fundamental issue of interweaving traffic from different applications. Moreover, their approach of forcing each session through multiple classifiers significantly reduces detection efficiency. As shown in Fig. 2, four classifiers in an enterprise network require each session to undergo four sequential detections. Under our experimental environment, a single model takes 0.5 ms per session, while four models accumulate to 2.1 ms, introducing latency that degrades user experience, especially with numerous classifiers. Thus, a system that can route each session to the appropriate learner efficiently is critical. This not only improves detection speed but also enables fine-grained identification of benign traffic by service type.

(ii) *The scarcity of labeled malicious samples hinders the detection of both known and emerging threats.* In an enterprise network environment, large volumes of normal business traffic (e.g., web browsing, file transfers, and emails) coexists with relatively small amounts of malicious attack traffic (e.g., DDoS, SqlInjection, and Botnet). This imbalance makes it challenging for detection models to effectively learn the characteristics of malicious traffic during training [16],

Received 13 January 2025; revised 2 August 2025; accepted 1 December 2025; approved by IEEE TRANSACTIONS ON NETWORKING Editor S. Lin. Date of publication 26 December 2025; date of current version 12 January 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62227805 and Grant 62572437 and in part by the Key Research and Development Program of Zhejiang Province under Grant 2023C01039. (Corresponding authors: Ziming Zhao; Fan Zhang.)

Peng Fang, Feiyang Huang, Ziming Zhao, and Fan Zhang are with the College of Computer Science and Technology, Zhejiang University, Hangzhou 310027, China (e-mail: fangpeng@zju.edu.cn; feiyanghuang@zju.edu.cn; zhaoziming@zju.edu.cn; fanzhang@zju.edu.cn).

Wenrui Ma is with the School of Computer Science and Technology, Zhejiang Gongshang University, Hangzhou 310018, China (e-mail: mawenrui@zjgsu.edu.cn).

Digital Object Identifier 10.1109/TON.2025.3648394

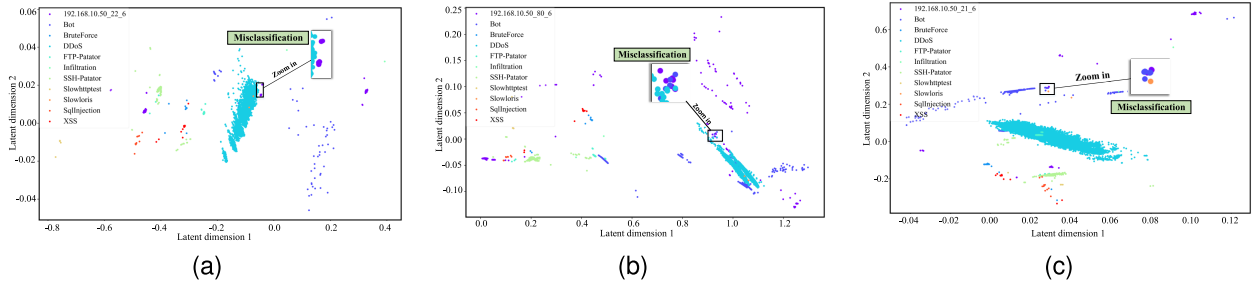


Fig. 1. Misclassification during training on single benign class and testing on different attack classes with dataset CICIDS2017. (a) Training with benign ssh traffic. (b) Training with benign web traffic. (c) Training with benign ftp traffic.

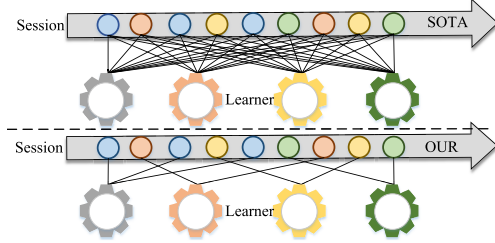


Fig. 2. Comparison of detection efficiency of one-class learners. Sessions with the same color come from the same service.

often resulting in high false positive (FP) rates for a small number of attack samples. Although oversampling and data augmentation schemes [17], [18] have been introduced to balance datasets across different categories, these methods typically rely on duplicating or synthesizing existing data, offering limited improvements in data diversity. Especially for unknown classes, previous solutions require large amounts of labeled malicious samples to achieve high detection accuracy [8], [19]. However, in real-world scenarios, emerging attacks such as zero-day exploits are rare and lack sufficient labeled samples for effective training. This scarcity of labeled attack samples limits the ability of conventional NIDS to adapt to emerging threats.

To address these challenges, we propose the novel **Service-Oriented Fine-Grained Attack traffic detection (SOFA)** framework, which redefines unknown attack detection as a two-stage task. In the first stage, a one-class anomaly detector identifies attacks using only benign traffic. To resolve traffic interweaving issues, SOFA adopts service-oriented one-class modeling based on service triplets.¹ It first identifies all service triplets in the enterprise network and trains independent one-class models for each, enabling accurate detection of attacks targeting specific services by learning their normal traffic patterns. The second stage uses a multi-classifier to classify isolated attacks and recognize emerging ones. Leveraging meta-learning with limited labeled samples, it utilizes a ResNet-based Siamese network, which is adept at capturing subtle distinctions in known attacks, to detect both familiar and new variants.

In summary, the contributions of this paper are four-fold:

- We carefully examine the problems of current anomaly-based NIDS in practice and summarize them as two key challenges. To facilitate addressing the above issues, we propose SOFA, a framework based on service-oriented

design and meta-learning for fine-grained traffic detection in few-shot scenarios.

- To overcome the limitations of coarse-grained modeling, we propose a one-class learner construction method for service triples, and develop a method to determine service triples through a pruning and reconstruction strategy of the session relationship graph, which does not depend on the characteristics of any protocol.
- To enhance the accuracy of fine-grained detection in few-shot scenarios, we propose applying Resnet to the Siamese network to construct a multi-classification learner. To mitigate the impact of noise on few-shot learning, we introduce a feature enhancement method, which effectively improves the model's detection accuracy.
- We implemented the prototype of SOFA and conducted extensive experiments using various real-world malicious traffic to verify its superior accuracy and efficiency compared to SOTA.

The rest of the paper is organized as follows: Section II reviews related works. Section III presents the problem formulation. Section IV introduces the high-level design of SOFA. In Section V, we describe the detailed designs. In Section VI, we experimentally evaluate the performances. Section VII provides an in-depth discussion of SOFA and points out its limitations. Section VIII concludes this paper.

II. RELATED WORK

This section mainly summarizes the NIDS technologies involved in this paper in terms of unknown attack detection and few-shot detection, as shown in Table I. The column of *Few-Shot Learning* indicates whether the model supports the few-shot learning paradigm; *Limited Labeled Samples* signifies whether the model achieves practical performance when there are ≤ 15 labeled samples per class; *Unknown Detection* indicates model's ability to identify unknown attacks; *Robust Detection* represents the performance stability under traffic obfuscation or noise injection; *Computational Efficiency* refers to whether the inference speed of the model ≤ 2 ms.

NIDS with Unknown Attack Detection. Unknown attack detection mainly includes unsupervised learning, semi-supervised learning methods. (i) Unsupervised methods. Many unsupervised learning methods have been proposed to identify outliers in network traffic, mainly using clustering methods [20], [21], graph association methods [14], [22] and encoder-decoder learning methods [23]. These methods are unable to identify malicious behavior in a fine-grained manner, and the

¹A service triplet $\langle ip, port, protocol \rangle$ uniquely identifies a server-side process (server-side socket).

TABLE I
COMPARISON OF DETECTION MODELS

Type	Model	Paper	Feature	Few-Shot Learning	Limited Labeled Samples(≤ 15)	Unknown Detection	Robust Detection	Computational Efficiency($\leq 2ms$)
NIDS with Unknown Detection	Clustering	[20] [21]	Statistics, header fields	×	✓	✓	×	×
		[19]	Flow-based Statistics	×	✓	✓	×	×
	Graph	[22] [14]	Flow-based Statistics	×	✓	✓	✓	×
	AutoEncoder	[23] [15]	Flow-based Statistics	×	✓	✓	×	✓
	Isolation Tree	[8] [24]	Flow-based Statistics	×	✓	✓	×	×
NIDS with Few-Shot Learning	Optimization	[25] [26] [27]	Statistics, header fields	✓	×	×	✓	✓
	Prototype	[28] [29] [30]	Packet raw bytes	✓	×	×	✓	✓
	Matching	[31] [32]	Statistics, header fields	✓	×	×	×	✓
	Relation	[33] [34] [35]	Statistics, raw bytes	✓	×	×	✓	✓
	Siamese	[36] [37]	Packet raw bytes	✓	✓	✓	×	✓
		[38]	Flow-based Statistics	✓	×	✓	×	✓
		This work	Statistics, raw bytes	✓	✓	✓	✓	✓

sparsity of benign traffic increases the difficulty of model training and fitting, resulting in a high false alarm rate. (ii) Semi-supervised methods. Semi-supervised methods (*e.g.*, Cls-Anomaly [15], FARE [19], FOSS [8]) combine unsupervised feature learning with limited labeled data. Although some methods have utilized the labels of few samples to improve detection accuracy, they lack a mechanism to extract meta-knowledge from limited labeled data, making it difficult to support few-shot generalization.

NIDS with Few-Shot Learning. Currently, meta-learning and its related algorithms have been applied to network intrusion detection and data analysis [34]. Most meta-learning-based intrusion detection methods are derived from traditional meta-learning approaches initially proposed in the field of computer vision, which can be primarily divided into two categories: optimization-driven methods and model-driven methods. Representative optimization-driven methods include Model-Agnostic Meta-Learning (MAML) [29], [39], while typical model-driven methods cover Rethinking Few-Shot (RFS) [40], Prototypical networks [41], Matching networks [42], Relation networks [43], and Siamese networks [44].

Optimization-driven. methods focus on parameter optimization to enable the model to quickly adapt to new few-shot tasks. Lu et al. [25] proposed a few-shot IoT intrusion detection model that combines CNNs for feature extraction and MAML for rapid parameter optimization. In order to enhance the incremental learning capability of the model, Davide et al. [27] combined RFS optimization with dynamic thresholds for incremental attack detection. Different from the optimization-driven methods, **Model-driven** methods rely on designing specific network structures or metric learning mechanisms to measure the similarity between samples in the feature space, thereby completing few-shot classification without heavy parameter adjustment. Huang et al. [31] improved the Matching networks by adding a gating mechanism to classify test samples as either known or unknown categories. Other works such as [32] and [45] have demonstrated the effectiveness of Matching networks in classifying IoT attack traffic. Xu et al. [34] proposed an end-to-end meta-learning framework that uses a feature extraction network F-Net and a comparison network C-Net to learn prior knowledge. Zheng et al. [33] proposed RBRN, a flow-based Relation network for encrypted traffic classification. Liang et al. [35] introduced OICS-VFSL, which optimizes intra- and inter-class structures for microservice intrusion detection. Yang et al.

[30] constructed a FS-IDS framework based on a Prototype network, encoding traffic into grayscale images and combining it with autoencoder feature fusion. Du et al. [28] introduced BFS-NID, which combines visual Transformer self-supervised pretraining and branch classifier learning modules to optimize prototype representations. Similarly, Chai et al. [29] proposed DPNSA, which enhances feature extraction using dynamic convolution and dual-sample activation functions. While these methods address detection and incremental model updates with few samples, they still need large training datasets for support sets and struggle to detect unknown categories like 0-day attacks. Different from the above method, Siamese networks learn the similarity between sample pairs, avoiding heavy reliance on large labeled training datasets for task-set building [46]. Chen et al. [36] developed a three-stage detection method that combines clustering and metric learning to achieve fine-grained detection of unknown behaviors. Yu et al. [37] applied TextCNN and one-hot encoding to construct metric models using Siamese networks for packet-level feature extraction. However, this method still suffers from the problem of false positives caused by sparse benign traffic.

To address these limitations, we propose a two-stage few-shot learning method. First, a service-oriented self-supervised model filters benign traffic, avoiding interference from its sparsity in few-shot classification. Second, a Siamese network is employed to identify known and unknown attacks. Experimental results show that our ResNet-based model achieves high detection accuracy with only a small number of attack samples.

III. PROBLEM FORMULATION

A. Threat Model and Assumptions

Adversary Model. This research focuses on external attacks targeting enterprise networks, excluding insider theft. We aim at sophisticated attackers using emerging tactics, including novel methods and variants of existing ones such as zero-day exploits, and address scenarios with scarce or no attack samples. Given the prevalence of encrypted networks (*e.g.*, SSL/TLS, SSH), we focus on encrypted traffic analysis. Instead of examining transmission content, we characterize traffic behavior via packet field distributions and patterns, enabling effective detection of encrypted malicious traffic.

Assumptions. We assume there is no prior knowledge when suffering unprecedented attacks in practice. Meanwhile, we

assume that we already know the range of internal IP addresses within the enterprise, such as 192.168.0.0/16, which means we know whether the IP address comes from internal or external sources. We do not assume that other Internet entities will carry out additional cooperation, such as the IP blocklist provided by the security provider. We assume that we have obtained normal traffic from some enterprise services and a small portion of labeled malicious behavior samples. In addition, when the server business changes, we assume that the victim can provide some legitimate traffic samples for model adaptation.

B. Design Goals and Problem Definition

The design goals of SOFA are as follows: First, it should be able to discover all open and externally accessible service triplets within the enterprise via traffic. Second, it can achieve universal detection to check if each service is under attack through encrypted or non-encrypted traffic, preventing attacks from evading detection via traffic encryption [47]. Third, it can accurately identify known attack types using a small number of labeled samples [34]. Finally, it can identify emerging attack behaviors like zero-day attacks that have not been disclosed [8]. These capabilities are supported by SOFA's two-stage design, and we precisely define the problems in the two stages below.

Service Aggregation and Anomaly Detection. Let $\mathcal{S}$ be the internal services. It can be represented as:

$$\mathcal{S} = \{s_i = (h_i, p_i, t_i) \mid h_i \in H, p_i \in P, t_i \in T\} \quad (1)$$

Among them, $i = \{1, \dots, v\}$, where v denotes the number of service triplets. H represents all IP addresses within the enterprise. P represents the business port of the open service (excluding random port), e.g., 21, 80, 443. T represents the transmission protocol used by the service. According to the connection relationship of network sessions, we can construct an adjacency matrix M , where the rows and columns represent the communication triplets. Thus, in the traffic screening stage, the goal is to design an algorithm $\mathcal{F}$ that can map the adjacency matrix M to a set of service triples $\mathcal{S}$, i.e., $\mathcal{F}(M) = \mathcal{S}$. After finding the $\mathcal{S}$, we use the internal benign traffic to construct a series of one-class classifiers $\mathcal{V} = \{\mathcal{V}_1, \mathcal{V}_2, \dots, \mathcal{V}_v\}$ for each $s_i \in \mathcal{S}$, so that the classifier can satisfy:

$$\mathcal{V}_i(x_i) = \begin{cases} 1, & \text{if } x_i \in \text{Attack} \\ 0, & \text{otherwise} \end{cases} \quad (2)$$

where x_i represents the feature vector of a session directed to the service triplet s_i .

Identification of Known and Unknown Attacks. Let $K = \{K_1, K_2, \dots, K_m\}$ be the set of labeled malicious attack classes, where m represents the number of known attack categories during training. Each class contains no more than l labeled samples for model training. This few-shot learning task is to design a model $\mathcal{M}$, such that:

$$\mathcal{M}(a_i) = \begin{cases} K_j, & \text{if } a_i \in K \\ U, & \text{if } a_i \notin K \end{cases} \quad (3)$$

where a_i denotes the test attack sample. U denotes the unknown attack class. The objective is to ensure that the model

can accurately classify attack samples into known classes while distinguishing them from unknown attack classes, even with limited labeled data.

IV. SOFA FRAMEWORK OVERVIEW

In this section, we depict a high-level workflow of SOFA, highlighting the essential components of each constituent element, as shown in Fig. 3. SOFA is a two-stage framework for service-oriented attack detection. In **Stage 1**, traffic is segregated by service triplets ($\langle ip, port, protocol \rangle$), and independent one-class models are trained to learn the benign traffic patterns for each triplet respectively. This isolates attack traffic while avoiding cross-service interference. In **Stage 2**, a meta-learning model classifies attacks using limited labeled samples. A Siamese network with feature enhancement constructs a discriminative metric space to compare attack behaviors. Known attacks are classified by similarity to labeled samples, while unknown attacks are identified by deviation from existing clusters. The output delivers fine-grained threat identification with minimal latency.

V. DESIGN DETAILS OF SOFA

A. Service Aggregation

In this section, we devised a service triplet identification approach predicated on a connectivity graph and pruned the coupled nodes through the computation of an adjacency matrix.

Service Graph Construction. When an enterprise provides external services, the normal traffic of the service is continuous. In order to reduce the complexity of graph construction, the traffic within 24 hours can be randomly sampled [48]. The sampled data can be reorganized into sessions. In order to eliminate the false connection behaviors in benign traffic, we use two conditions to filter the sampled sessions: the number of frames in the session is greater than 3 and the session is a bidirectional flow. The session index can be formalized as a connection relationship in the graph, with two vertices represented as $\langle srcip, srcport, protocol \rangle$ and $\langle dstip, dstport, protocol \rangle$. The edge represents the session from the source triplet to the destination triplet. The undirected graph constructed in this way will show obvious aggregation characteristics.

Pruning and Reconstruction. Service triplets are determined by setting an appropriate degree threshold. In SOFA, we empirically set the degree threshold to 2. However, in actual networks, not all triplets with a degree greater than 1 are service triplets. This is because some clients may use the same random port to communicate in different sessions, resulting in the degree of the client triplets exceeding 1. In order to solve the problem of random port coupling, we implement edge pruning and edge adding operations. Fig. 4 illustrates the edge deletion and addition process.

Fig. 4a describes the communication relationship under normal communication conditions. The green dots represent client triples and the yellow dots represent server triples. Multiple client triples may access the same server triplet. Fig. 4b illustrates the situation where the client uses the same random port to access different service triplets. Coupled client

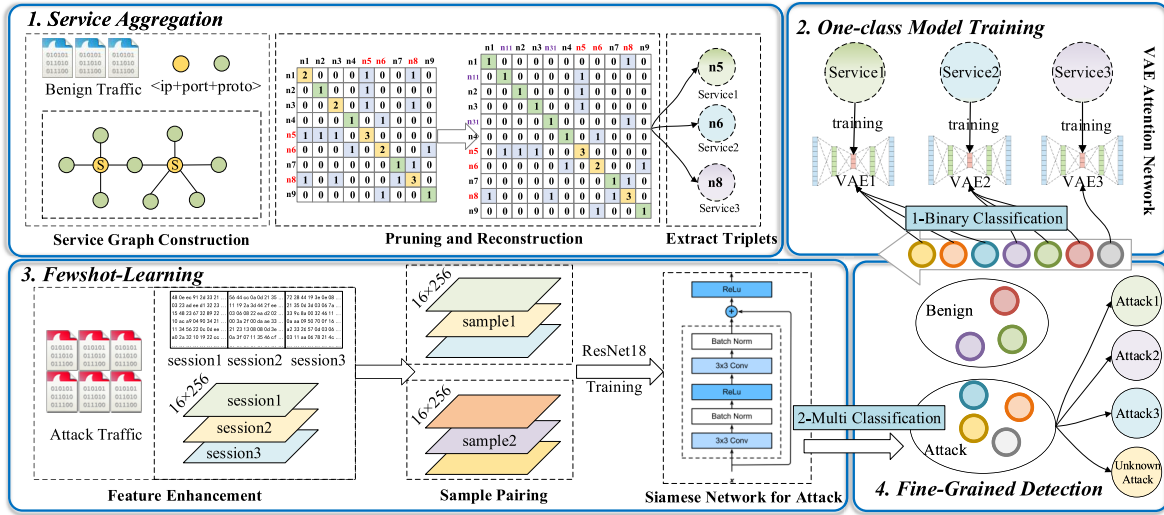


Fig. 3. The overview of SOFA. It consists of four parts: Service aggregation, one-class model training, fewshot-learning and fine-grained detection.

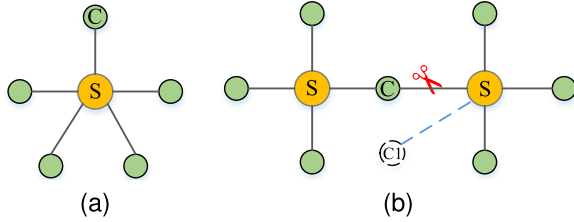


Fig. 4. The process of graph pruning and reconstruction.

	n1	n2	n3	n4	n5	n6	n7	n8	n9
n1	2	0	0	0	1	0	0	1	0
n2	0	1	0	0	1	0	0	0	0
n3	0	0	2	0	1	0	0	1	0
n4	0	0	0	1	0	1	0	0	0
n5	1	1	1	0	3	0	0	0	0
n6	0	0	0	1	0	2	0	0	1
n7	0	0	0	0	0	0	1	1	0
n8	1	0	1	0	0	0	1	3	0
n9	0	0	0	0	0	1	0	0	1

	n1	n11	n2	n3	n31	n4	n5	n6	n7	n8	n9
n1	1	0	0	0	0	0	0	0	1	0	0
n11	0	1	0	0	0	0	1	0	0	0	0
n2	0	0	1	0	0	0	1	0	0	0	0
n3	0	0	0	1	0	0	1	0	0	0	0
n31	0	0	0	0	1	0	0	0	0	1	0
n4	0	0	0	0	0	1	0	1	0	0	0
n5	0	1	1	1	0	0	3	0	0	0	0
n6	0	0	0	0	0	1	0	2	0	0	1
n7	0	0	0	0	0	0	0	0	1	1	0
n8	1	0	0	0	1	0	0	0	1	3	0
n9	0	0	0	0	0	0	0	1	0	0	1

Fig. 5. The process of pruning and rebuilding edges for coupling nodes. n_i represents a triplet. The values in the adjacency matrix represent the degrees of the vertices.

triples can easily be misunderstood as service triples. We need to cut off the coupled edges and randomly generate ports between 49152 and 65535 on the coupled nodes to re-establish the edges.² We implement the whole process by establishing an adjacency matrix. Fig. 5 depicts the process of employing an adjacency matrix to search for service triplets. Herein, n_i represents the triplet, which serves as a vertex in the graph and encompasses both the client triplet and the server triplet. Nodes n_5 , n_6 and n_8 denote the actual service triplets, while other nodes represent client triplets. $d(n_i, n_j)$, where $i \neq j$, indicates whether a communication relationship exists between triplet n_i and triplet n_j . The diagonal elements $d(n_i, n_i)$ of the adjacency matrix signify the number of connected neighbors

within a triplet n_i .

$$d(n_i, n_i) = \sum_j d(n_i, n_j), i \neq j \quad (4)$$

As can be seen from the left matrix in Fig. 5, when the preset threshold 2 is exceeded, they are all considered as service triplets. Consequently, n_1 , n_3 , n_5 , n_6 and n_8 in the matrix are all considered as candidate sets of service triplets. However, in practice, n_1 and n_3 both have coupled ports because they use the same random port in different sessions. To eliminate this coupling phenomenon, we define the following conditions for coupling ports:

- (i) The total degree of the current node is greater than threshold, $d(n_i, n_i) \geq 2$.
- (ii) The current node has the smallest degree among all neighboring nodes with communication relationships, $d(n_i, n_i) = \min_j \{d(n_j, n_j) \mid d(n_i, n_j) \neq 0\}$.

Based on the above conditions, the coupling nodes n_1 and n_3 can be determined. According to the coupling neighbor relationship, the neighbor of node n_i can be expressed as $N_i = \{n_j \mid d(n_i, n_j) \neq 0, i \neq j\}$. Randomly select a neighbor node $n_k = \text{random}_j \{n_j \mid n_j \in N_i\}$ and retain the edge $e(n_i, n_k)$ with this neighbor. The edges between other neighbors can be deleted, and a new node can be constructed by randomizing a port while deleting the edges, with edges added between the new node and the old neighbor. In the figure, n_1 is a coupled node. Randomly select the neighbor n_8 as the reserved node, preserve the neighbor relationship $e(n_1, n_8)$, delete the edges with other neighbors $e(n_1, n_5)$, and then generate a new node n_{11} by adding edges between n_{11} and n_5 . Repeat the process for n_3 , the entire adjacency matrix will be transformed into the matrix on the right side of the figure. At this point, it can be ascertained that n_5 , n_6 and n_8 are the service triplets we need to search for. The above method may lead to discrimination errors under the following two extreme circumstances: a client uses the same random port to access multiple inactive³ services; an inactive service

²The ports in the range [49152, 65535] are defined as dynamic random ports by RFC6335.

³In this paper, inactive services refer to those that receive limited user requests within the sampling period.

is accessed only once within the sampling period. These two situations can be alleviated by model updating, as discussed in Section VII-C.

Extract Triplets. Graph pruning and reconstruction are followed by identifying service triplets: if diagonal elements in the adjacency matrix exceed the preset minimum degree threshold, they are determined as service triplets. This method is also applicable to services that use random but controllable ports (*e.g.*, FTP passive mode). In large enterprises with hundreds of services, two optimization strategies improve efficiency and reduce resource consumption: (i) Parallel training and deployment. Training samples for different services are isolated, enabling parallel training across machines. After training, the VAE model for each service is deployed to its corresponding terminal server. For terminals with multiple services, a session routing system using hash mapping links sessions to models for efficient parallel detection. High-concurrency services use multiple identical model instances across servers, with dynamic session distribution via a service registration center and load balancing to ensure stable loads. (ii) Model sharing and service clustering. Many service triplets in large enterprises handle identical business. For such cases, a single VAE model is sufficient for training purposes. During deployment, multiple triplets can either share this model at an aggregation center (a dedicated detection server) or utilize copied models on their respective terminals, thereby reducing server memory consumption. Aggregation can be based on traffic characteristic similarity (*e.g.*, [15], [49]), identical protocols and ports, or manual configuration by administrators.

B. One-Class Model Training

Feature Extraction. In this section, we aggregate all sessions indexed by the 5-tuple $\langle srcip, srcport, dstip, dstport, protocol \rangle$ into the corresponding service triplet buckets. We mainly focus on two transport protocols, namely TCP and UDP. When using the UDP transport protocol, some TCP feature items in the table will be filled with 0, such as TCP flag features SYN, ACK, *etc.* These features characterize the traffic in terms of temporal, volumetric, and header-field distributions [8]. Specifically, the feature vector roughly consists of three parts: (i) the protocol number; (ii) the count of a series of flags from layers 1-4 in three cases (forward, backward, and bidirectional), totaling 33/15 (TCP/UDP) dimensions; (iii) the 72/36-dimensional statistics of each session in three directional situations (*i.e.*, maximum, minimum, mean, and standard deviation). Generally, all features are of either int or float types.

Model Architecture. In the training of a one-class model for anomaly detection, we adopt a Variational AutoEncoder (VAE) architecture based on an attention mechanism. VAE is a probabilistic generative model that learns to compress and reconstruct input data. The overall architecture of the attention-based VAE model is presented in Fig. 6, which is formally defined as follows.

(i) Encoder network. The encoder is composed of two fully connected (dense) layers. The first layer maps the input

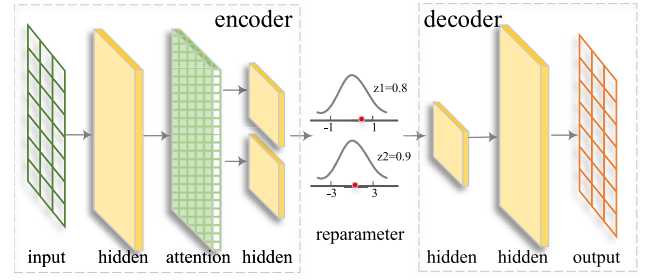


Fig. 6. VAE model based on attention.

features to a hidden space:

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \quad (5)$$

An attention mechanism is applied to the hidden representation $\mathbf{h}_1$:

$$\text{attn_weights} = \text{Sigmoid}(\mathbf{W}_{\text{attn}} \mathbf{h}_1 + \mathbf{b}_{\text{attn}}) \quad (6)$$

$$\mathbf{h}'_1 = \mathbf{h}_1 \odot \text{attn_weights} \quad (7)$$

The output of the encoder is then passed through the second layer to produce the mean and log variance of the latent space:

$$\mu, \log(\sigma^2) = \mathbf{W}_2 \mathbf{h}'_1 + \mathbf{b}_2 \quad (8)$$

(ii) Latent space. The latent space is parameterized by the mean μ and the log variance $\log(\sigma^2)$. The reparameterization trick is used to sample from the latent space:

$$\mathbf{z} = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \quad (9)$$

(iii) Decoder network. The decoder reconstructs the input from the latent space:

$$\mathbf{h}_3 = \text{ReLU}(\mathbf{W}_3 \mathbf{z} + \mathbf{b}_3) \quad (10)$$

$$\mathbf{x}' = \text{Sigmoid}(\mathbf{W}_4 \mathbf{h}_3 + \mathbf{b}_4) \quad (11)$$

(iv) Loss function. The VAE is trained using a loss function that combines the reconstruction loss, the Kullback-Leibler divergence (KLD), and a sparsity penalty:

$$\mathcal{L} = \mathcal{L}_{\text{MSE}} + \mathcal{L}_{\text{KLD}} + \lambda \mathcal{L}_{\text{sparsity}} \quad (12)$$

$$\mathcal{L}_{\text{MSE}} = \sum_{i=1}^{\text{input_dim}} (\mathbf{x}'_i - \mathbf{x}_i)^2 \quad (13)$$

$$\mathcal{L}_{\text{KLD}} = -\frac{1}{2} \sum_{i=1}^{\text{latent_dim}} (\exp(\log(\sigma_i^2)) + \mu_i^2 - 1 - \log(\sigma_i^2)) \quad (14)$$

$$\mathcal{L}_{\text{sparsity}} = \sum_{i=1}^{\text{latent_dim}} |\mu_i| \quad (15)$$

Anomalies Detection. In this section, we develop some outlier identification schemes such as the preset method and extreme value theory method.

(i) Preset method. We establish a threshold based on the α^{th} percentile of reconstruction errors for each model. The α^{th} percentile, denoted as Q_α , is calculated from the set of reconstruction errors $\{e_i\}$, where e_i represents the error for

the i -th training data point. The process is formally defined as follows:

$$Q_\alpha = \min \left\{ e_i \in \{e_1, e_2, \dots, e_n\} \mid \frac{1}{n} \sum_{j=1}^i \mathbb{I}_{\{e_j \leq e_i\}} \geq \alpha \right\} \quad (16)$$

Here, n is the total number of training data points, and $\mathbb{I}_{\{e_j \leq e_i\}}$ is the indicator function which equals 1 if $e_j \leq e_i$ and 0 otherwise. Then, anomalies are identified as data points whose reconstruction error exceeds the predefined threshold T .

$$\text{recon_error} = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_i)^2 > T = Q_\alpha \quad (17)$$

where n is the number of features in the data, x_i and $\hat{x}_i$ are the i -th features of the input and reconstructed data, respectively. The choice of α in Equation (16) is affected by the specific network traffic distribution. When the dataset contains significant noise, a relatively small threshold is recommended, such as $\alpha = 0.9$. Conversely, for noise-free datasets with distinct boundaries between different categories, a larger threshold should be adopted, such as $\alpha \in [0.95, 0.99]$.

(ii) Extreme Value Theory [50]. In this section, we employ the Extreme Value Theory (EVT) to devise a more robust method for identifying outliers, a technique that has also been widely applied in [14] and [51]. The fundamental concept in EVT is the Extreme Value Distributions (EVD).

For a random variable X with cumulative distribution function $F(x) = \mathbb{P}(X \leq x)$, the tail of the distribution is denoted as $\bar{F}(x) = 1 - F(x) = \mathbb{P}(X > x)$. The EVD has the form:

$$G_\gamma : x \mapsto \exp \left(-(1 + \gamma x)^{-\frac{1}{\gamma}} \right), \gamma \in \mathbb{R}, 1 + \gamma x > 0 \quad (18)$$

where $\gamma \in \mathbb{R}$ denotes the extreme value index, and $1 + \gamma x > 0$. The extreme value index γ determines the shape of the tail and is related to the original distribution of the data.

The Peaks-Over-Threshold (POT) approach is a key component of EVT theory. Based on the Pickands-Balkema-de Haan theorem (also called second theorem in EVT) [52], the excess over a threshold t , denoted as $X - t$, follows a Generalized Pareto Distribution (GPD) with parameters γ and σ . This allows us to estimate the distribution of extreme values and calculate the threshold z_q such that $\mathbb{P}(X > z_q) < q$. The threshold z_q can be computed as

$$z_q \simeq t + \frac{\hat{\sigma}}{\hat{\gamma}} \left(\left(\frac{qn}{N_p} \right)^{-\hat{\gamma}} - 1 \right) \quad (19)$$

where q is the risk parameter, n is the total number of observations, and N_p is the number of peaks (observations greater than t). To estimate the parameters $\hat{\gamma}$ and $\hat{\sigma}$ of the GPD, we use the maximum likelihood estimation method. The likelihood function is given by

$$\log \mathcal{L}(\gamma, \sigma) = -N_p \log \sigma - \left(1 + \frac{1}{\gamma} \right) \sum_{i=1}^{N_p} \log \left(1 + \frac{\gamma}{\sigma} \mathcal{P}_i \right) \quad (20)$$

where $\mathcal{P}_i = X_i - t$ for $X_i > t$. To solve this optimization problem, we can use the Grimshaw's trick, which reduces

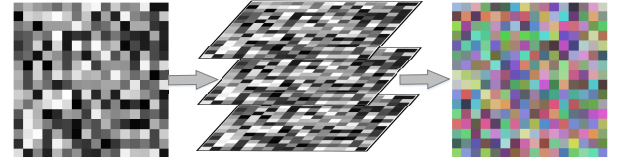


Fig. 7. The process of feature enhancement. Stack and combine the two sessions before and after each session into a three channel color image.

the two-variable optimization to a single-variable equation. By finding the roots of the equation $u(x)v(x) = 1$, where

$$u(x) = \frac{1}{N_p} \sum_{i=1}^{N_p} \frac{1}{1 + x\mathcal{P}_i}, \quad v(x) = 1 + \frac{1}{N_p} \sum_{i=1}^{N_p} \log(1 + x\mathcal{P}_i) \quad (21)$$

we can retrieve $\hat{\gamma}$ and $\hat{\sigma}$. Finally, the estimated values $\hat{\gamma}$ and $\hat{\sigma}$ can be substituted into Equation (19) to calculate the threshold.

The advantage of presetting thresholds is that they are easy to implement and suitable for environments with relatively stable traffic patterns. Many models have verified their effectiveness [53], [54], [55], [56]. However, their limitation is that they require manual setting of threshold ratios and lack flexibility, especially when facing dynamically changing traffic characteristics. The advantage of the EVT method lies in its theoretically supported automatic threshold derivation capability, which is particularly suitable for traffic detection in dynamic network environments. The EVT method is more suitable for scenarios with complex traffic patterns and significant concept drift, and can dynamically adapt thresholds based on sample distribution.

C. Few-Shot Learning

Feature Enhancement. The features used in this section are different from those used in the earlier training of the VAE model. Due to the limited sample size, more attention needs to be paid to the features of byte granularity within small samples. Initially, it is necessary to restructure the session based on the 5-tuple index. Specifically, we set the number of packets in each session to $M_1 = 16$ and the number of bytes to $N_1 = 256$ in each packet. To elaborate, we parse each pcap file and extract the first 256 bytes of each packet, starting from the Ethernet header. Random anonymization has been applied to MAC addresses, IP addresses, and port numbers [57]. In fact, this feature extraction method remains effective for encrypted traffic. The byte feature matrix contains key information from IP headers, TCP handshake phases, and certificate exchange stages (e.g., those in SSH, HTTPS, and RDP protocols), enabling it to effectively capture the differences between various malicious behaviors [10], [29], [34].

In N -way K -shot small sample learning scenarios, the value of K is usually relatively small, such as 5, 10, or 15. When dealing with a limited sample size, the presence of noise within the samples can have a severely detrimental effect on the model. This may lead the model to erroneously learn the characteristics of the noise, thereby resulting in inaccurate subsequent recognition. For instance, in widely utilized public datasets like CICIDS2017 [58] and Botnet2014 [59], noise occurrences are frequently observed. To mitigate the impact of noise on small sample training sets, we employed

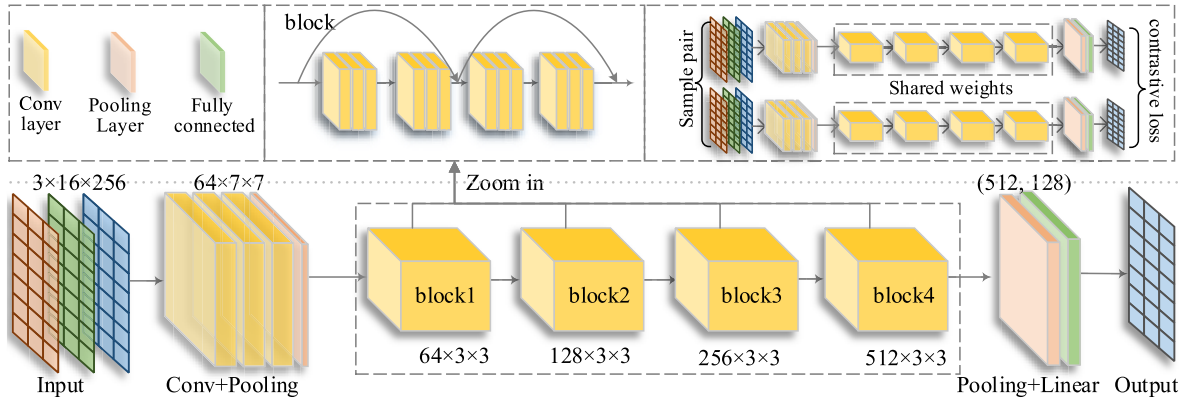


Fig. 8. Siamese network architecture based on ResNet18. The upper right area shows the model training process using sample pairs. The area below shows the design details of the Siamese network.

feature enhancement techniques to eliminate the influence of such noise. The process of feature enhancement is shown in Fig. 7. The whole process converts the original 2-dimensional grayscale image into a 3-dimensional color image.

For each sample X_i with label y_i , we first find other samples from the same service triplet and with the same client IP. If there are multiple samples, we shuffle the sample order and choose the current node X_i , the previous node X_{i-1} , and the next node X_{i+1} to jointly form the new feature matrix X'_i :

$$X'_i = \begin{bmatrix} X_{i-1} \\ X_i \\ X_{i+1} \end{bmatrix} \quad (22)$$

where $X_i, X_{i-1}, X_{i+1} \in \mathbb{R}^{1 \times 16 \times 256}$. If a session is unique within its service triplet, we replicate the session three times to maintain the three-channel structure.

This results in a new dataset where each sample is represented by three channels of network sessions. It enriches the feature space and enables the model to learn from the temporal context of network activity, thereby mitigating the impact of noise on small sample training sets. We discuss the effectiveness of feature enhancement in Section VI-G.

Sample Pairing. In the process of training Siamese network to perform network intrusion detection tasks, a key step is to construct a dataset containing sample pairs. These pairs are designed to reflect the similarity or dissimilarity between samples, which is fundamental for the network to learn distinguishing features. This process involves the generation of pairs from the dataset X and their corresponding labels y , with the aim of enhancing the model's ability to discern between intra-class and inter-class variations. For a given class $\mathcal{X}_i$, the intra-class pairs can be expressed as $\mathcal{I}_{ntra}^i = \{(X_j, X_k) \mid X_j, X_k \in \mathcal{X}_i, j \neq k\}$. The number of such pairs is given by the binomial coefficient $\binom{n_i}{2}$, where n_i is the number of samples. Each pair is assigned 1 to indicate that they belong to the same class. For two distinct classes $\mathcal{X}_i$ and $\mathcal{X}_j$, the inter-class pairs can be presented as $\mathcal{I}_{nter}^{ij} = \{(X_{i1}, X_{j1}) \mid X_{i1} \in \mathcal{X}_i, X_{j1} \in \mathcal{X}_j\}$. Each pair is labeled as 0 to indicate that they belong to different classes.

Siamese Network Architecture. In the realm of deep learning, particularly within the confines of small sample sizes, the choice of model architecture plays a pivotal role in the efficiency and effectiveness of learning. We choose

ResNet18 as the base model of our Siamese network for the following reasons: (i) Parameter efficiency. Smaller models like ResNet18 are less prone to overfitting due to their reduced capacity, which is crucial when the amount of training data is limited. (ii) Computational efficiency. With fewer parameters, ResNet18 requires less computational power and training time compared to larger models, making it suitable under constrained time and resource availability. (iii) Feature extraction. ResNet18's residual architecture is adept at learning hierarchical features, which is essential for capturing the nuances within small datasets. The architecture is detailed in Fig. 8.

The architecture is designed to learn a metric space where the distance between feature representations corresponds to the similarity between input samples. The network accepts input dimensions of $2 \times 3 \times 16 \times 256$ for each pair of images. Each branch of the Siamese network consists of the ResNet18 base model, which includes a series of convolutional layers, batch normalization, and ReLU activations. The feature maps are progressively reduced in spatial dimensions while increasing in depth. The output of both branches is a 128-dimensional vector.

The contrastive loss function is employed to train the Siamese network, aiming to minimize the distance between similar pairs and maximize the distance between dissimilar pairs. The contrastive loss is defined as:

$$L = \frac{1}{N} \sum_{i=1}^N [y_i \cdot d(\mathbf{a}_i, \mathbf{b}_i)^2 + (1 - y_i) \cdot \max(0, m - d(\mathbf{a}_i, \mathbf{b}_i))^2] \quad (23)$$

where y_i is an indicator function used to indicate whether the sample pair $(\mathbf{a}_i, \mathbf{b}_i)$ is similar or dissimilar. m is the margin which can help distinguish similar and dissimilar samples and affects the loss calculation for negative pairs to separate them better in the feature space. $d(\mathbf{a}_i, \mathbf{b}_i)$ is the Euclidean distance between the feature vectors. The loss function is backpropagated through the network to compute the gradients, which are then used by the optimizer to update the weights of the network, thereby learning the metric space.

This method enables the Siamese network to mine subtle features of the sample space and accurately measure the similarity of input pairs.

D. Fine-Grained Attack Detection

Known Attacks Identification. This section presents a fine-grained attack detection method based on the Siamese network. It utilizes the feature extraction ability of the ResNet-based Siamese network to compute the average feature representation of each category after training. During inference, the feature representation of the test sample is compared with the pre-computed category averages via a distance metric to classify it into the proper class.

To enable fine-grained classification, we compute a mean feature vector μ_c for each class c in the training dataset. These mean vectors serve as reference benchmarks in the feature space. Given a class c having training samples $S_c = \{x_i \mid y_i = c\}$, the mean feature vector is defined as:

$$\mu_c = \frac{1}{|S_c|} \sum_{x_i \in S_c} f(x_i) \quad (24)$$

where $f(x_i)$ is the 128-dimensional feature vector extracted by the Siamese network. $|S_c|$ is the number of samples in class c . During inference, a test sample x_{test} is assigned to the class c^* with the smallest distance between its feature representation $f(x_{test})$ and the precomputed class means $\{\mu_c\}_{c=1}^C$:

$$c^* = \arg \min_c d(f(x_{test}), \mu_c) \quad (25)$$

here $d(\cdot, \cdot)$ denotes the Euclidean distance. This process ensures that each test sample is classified based on its proximity to the reference class means in the feature space.

Unknown Attacks Detection. In the context of unknown attack detection, the goal is to correctly identify known categories while accurately distinguishing unknown attacks that are not seen during training. However, the following challenges arise: (i) Small sample size. The training dataset contains very few samples, leading to unreliable estimation of decision boundaries. (ii) Concept drift. The data distribution in the test set may differ from the training set, especially in the case of unknown attacks. The EVT theory mentioned above has lost its advantage under small sample conditions. To address these challenges, we propose a global + local adaptive threshold adjustment method. The local threshold is set to adapt to the distribution of training samples of the current category. The global threshold is obtained based on the distribution of the minimum and maximum threshold intervals of each category to prevent overfitting.

(i) Local adaptive threshold. The local adaptive threshold is set based on the distribution of distances between training samples and their corresponding class centers. For each class c , we need to calculate the mean distance $\mu_c^d = \mathbb{E}[d(f(x_i), \mu_c)]$ and standard deviation $\sigma_c = \sigma[d(f(x_i), \mu_c)]$ to capture the distribution characteristics of the distances within the class.

To mitigate the influence of outliers, we defined lower and upper bound as:

$$l_c = \mu_c^d - \alpha \cdot \sigma_c, \quad u_c = \mu_c^d + \alpha \cdot \sigma_c \quad (26)$$

Distances that satisfy $d < l_c$ or $d > u_c$ are considered as noise samples and are excluded. Using the filtered distances, the mean and standard deviation are recalculated to establish a more refined measure of central tendency and dispersion. The

function then calculates an adapted threshold τ_c for each class c by incorporating the z -score at the desired confidence level:

$$\tau_c = \mu_c^d + z \cdot \sigma_c, \quad z = \Phi^{-1} \left(1 - \frac{\beta}{2} \right) \quad (27)$$

where β is the significance level, $0 < \beta < 1$. Φ^{-1} is the inverse of the cumulative distribution function (CDF) [60] of the standard normal distribution. The cumulative distribution function $\Phi(x)$ of the standard normal distribution is:

$$\Phi(x) = \frac{1}{\sqrt{2\pi}} \int_{-\infty}^x e^{-\frac{t^2}{2}} dt \quad (28)$$

There is no simple analytical expression for Φ^{-1} . Here we use Newton-Raphson iteration method [61] to approximate the solution. Let $y = 1 - \frac{\beta}{2}$. The Newton-Raphson iteration method is used to solve for the x -value (i.e., z -value) such that $\Phi(x) = y$. The iteration formula is:

$$x_{n+1} = x_n - \frac{\Phi(x_n) - y}{\varphi(x_n)} \quad (29)$$

where $\varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{x^2}{2}}$ is the probability density function of the standard normal distribution. The initialization x_0 can be set as $x_0 = \sqrt{-2 \ln(y)}$. Substituting the approximate solution x^* into the Equation (27), we can obtain the local threshold τ_c for each category.

(ii) Global lower bound. Furthermore, an empirically determined global threshold τ'_c is selected to prevent the phenomenon of lower thresholds caused by overfitting. We will discuss the selection of τ'_c in depth in Section VII-B. The final threshold for each class is determined by following equation:

$$\tau_c = \max(\tau_c, \tau'_c) \quad (30)$$

VI. EVALUATION

In this section, we implement the prototype of SOFA and comprehensively evaluate its effectiveness in terms of supervised evaluation, binary classification anomaly detection, multi-classification of attacks under few-shot settings, unknown attack detection with various proportions, and ablation study.

A. Experiment Setup

Testbed. The implementation of our experiments was executed on a server running CentOS Linux 7. Our program was carried out on NVIDIA GeForce RTX 4090 GPUs. Python 3.7.12 was selected as the programming language, and the PyTorch framework was utilized to enhance the functionality and efficiency of our experimental procedures.

Datasets. A series of public datasets are used for evaluation.

(i) **CICIDS2017** [58] datasets uses a proposed B-Profile system to generate the normal traffic, meanwhile, a series of intrusion behaviors are involved, including *BruteForce*, *DDoS*, *etc.* (ii) **USTC-TFC** [62] dataset involves network traffic from two parts, i.e., 10 types of malware and 10 categories of benign applications. (iii) **CICDOS2017** [63] dataset carries 24 hours of network traffic simulated on a testbed of application-layer *DoS* attacks intermixed with attack-free traces. This dataset presents 8 *DoS/DDoS* application-layer attacks.

TABLE II
REC AND F1 SCORE OF MODELS ON CICIDS2017 WITH FEW SAMPLES

Model	CICIDS2017															
	Benign		XSS		DDoS		SqlInjection		FTP-Patator		BruteForce		Botnet		SSH-Patator	
	Rec	F1	Rec	F1	Rec	F1	Rec	F1	Rec	F1	Rec	F1	Rec	F1	Rec	F1
ET-BERT	0.8910 0.0123	0.8083 0.0140	0.9333 0.0085	0.9081 0.0120	0.7582 0.0207	0.8351 0.0169	0.9411 0.0098	0.8205 0.0164	0.7412 0.0218	0.8093 0.0131	0.9056 0.0104	0.8421 0.0167	0.8614 0.0142	0.8866 0.0150	0.9073 0.0097	0.8194 0.0125
FOSS	0.7926 0.0191	0.8492 0.0126	0.9250 0.0081	0.9227 0.0091	0.8506 0.0130	0.8738 0.0142	0.7750 0.0165	0.8368 0.0162	0.7952 0.0203	0.8753 0.0142	0.8420 0.0173	0.8776 0.0180	0.7250 0.0234	0.8135 0.0147	0.8255 0.0123	0.8662 0.0165
Trident	0.9188 0.0087	0.9005 0.0111	0.9080 0.0099	0.8615 0.0149	0.8381 0.0154	0.9119 0.0097	0.8662 0.0148	0.9282 0.0099	0.7240 0.0224	0.8399 0.0157	0.8675 0.0162	0.9291 0.0116	0.6200 0.0172	0.7506 0.0203	0.8980 0.0149	0.9248 0.0086
FlowPic	0.8300 0.0158	0.7676 0.0175	0.5350 0.0235	0.6971 0.0178	0.8050 0.0139	0.8530 0.0139	0.8925 0.0131	0.9432 0.0088	0.9500 0.0106	0.7038 0.0232	0.9425 0.0120	0.7757 0.0200	0.5651 0.0190	0.7197 0.0191	0.8225 0.0127	0.9026 0.0094
I ² RNN	0.7174 0.0339	0.7125 0.0412	0.7971 0.0439	0.7957 0.0343	0.7202 0.0375	0.8081 0.0130	0.7885 0.0389	0.7935 0.0445	0.8468 0.0136	0.9099 0.0086	0.7336 0.0377	0.7603 0.0446	0.9303 0.0088	0.9181 0.0093	0.9587 0.0058	0.8772 0.0136
SOFA	0.9860 0.0055	0.9930 0.0041	1.0000 0.0000	1.0000 0.0000	1.0000 0.0000	1.0000 0.0000	1.0000 0.0000	0.9565 0.0040	1.0000 0.0000	1.0000 0.0000	1.0000 0.0000	1.0000 0.0000	0.9992 0.0039	0.9996 0.0038	1.0000 0.0000	1.0000 0.0000

[†] The smaller red fonts denote the standard deviation.

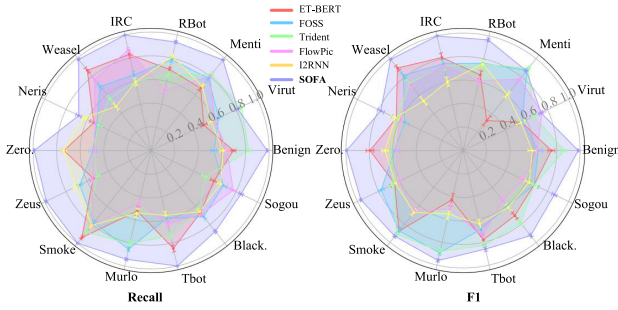


Fig. 9. Average Rec and F1 score with standard deviation on Botnet2014 under few samples.

(iv) **Botnet2014** [59] dataset was constructed by merging data traces. In this dataset, botnet and benign data were replayed via TCPReplay and captured by TCPdump as one unified dataset, with the overlay method used for this synthesis. It contains 14 different types of botnets. Due to the presence of substantial noise in the collection and labeling process of the dataset, we referred to reference [64], [65], [66] to perform denoising on the dataset.

Baselines. We use 17 models (including supervised, unsupervised and few-shot models) as baselines, covering SOTA methods in the traffic analysis landscape. The models are **ET-BERT** [12], **Kitsune** [49], **FOSS** [8], **Trident** [14], **HyperVision** [22], **FARE** [19], **FlowPic** [67], **Whisper** [6], **I²RNN** [10], **TrafficScope** [68], few-shot learning method **FC-Net** [34], **FS-IDS** [30], **CNN-MAML** [25], **RFS** [27], **RBRN** [33], **PBCNN** [37] and **KNN-Siamese** [38]. Tailored to the unique characteristics of diverse models and the specific conditions of the datasets at hand, we meticulously designed and executed a comprehensive series of comparative experiments across four distinct datasets.

Parameter Settings. The hyperparameter min_degree used to determine the service triplets during graph construction is set to 2. The hyperparameters used in VAE training are as follows: $lr=1e-3$, $batch_size=64$, $input_dim=106$, $epochs=100$, $hidden_dim=128$, $latent_dim=16$, t and q used in Equation (19) are set to 0.95 and $5e-2$. The hyperparameters used in the Siamese network training phase are as follows: $epochs=30$, $batch_size=10$, $lr=1e-4$, $\beta=0.95$ used in Equation (27) and α in Equation (26) is set to 3. The τ'_c in Equation (30) used during the evaluation was set to 0.1.

Metrics. We mainly use Accuracy (ACC), Precision (Pre), Recall (Rec), and F1 score because they are the most widely used in the literature [12], [49], [69] and [70].

B. Supervised Evaluation

To evaluate SOFA's comprehensive performance, we conducted supervised assessments on two datasets, assuming sufficient benign samples but limited attack samples (≤ 15 per malicious category). To mitigate class imbalance impacts on baseline models, we repeated sampling for classes with fewer samples to minimize overfitting. Results are in Table II and Fig. 9, using Recall and F1 to focus on category-wise recognition under imbalance. For CICIDS2017 [58], 8 classes were selected for comparison with SOTA methods; others were excluded due to insufficient samples (e.g., *Infiltration*) or mismatched SOTA prerequisites (e.g., *Portscan* for ET-BERT [12]). Similarly, 14 categories were chosen from Botnet2014 [59].

As can be seen from Table II, our method performs high accuracy in all categories of detection, and the Rec and F1 are close to 100% in most malicious behavior detection, which is significantly better than other methods. The performance table includes the standard deviation (in smaller red font) and the mean. SOFA incorrectly classified a limited number of *Botnet* attack samples as *SqlInjection*, thus resulting in some discrepancies between these two categories. In Fig. 9, we plotted a comparison chart based on the average Rec and F1 of each category and the corresponding standard deviation (std). The purple area represents our method. The closer the covered area is to the entire circle, the better the method performs. It can be seen that our method has achieved high Rec and F1 values in almost all categories. Overall, the classification performance of SOFA for known classes is competitive compared to supervised SOTA methods.

C. Comparison With Few-Shot Learning Models

We comprehensively compared our method with few-shot learning models in terms of detection accuracy and computational efficiency, conducting multi-classification experiments on the CICIDS2017 [58] and Botnet2014 [59] datasets. With sufficient benign training samples but limited attack samples (5, 10, 15 per category), results are shown in Fig. 10 and Fig. 11, using hyperparameters as in original papers. Notable

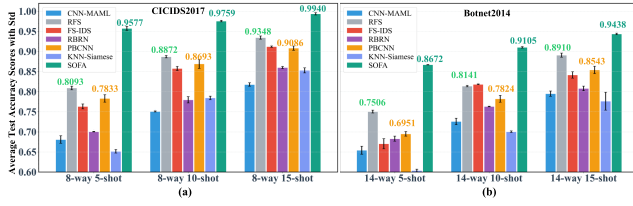


Fig. 10. Test accuracy scores of different FSL models.

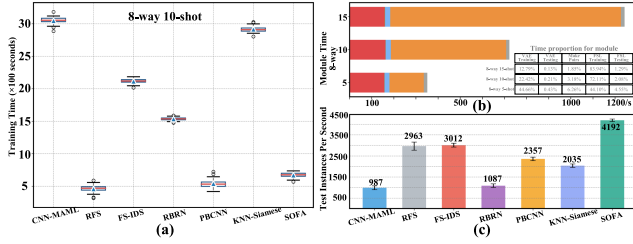


Fig. 11. Time overhead of different FSL models.

baselines encompass KNN-Siamese [38], which applies KNN to benign traffic, and RBRN [33], which employs data augmentation for attack traffic. Episode-based methods [27], [29] sample benign traffic as usual and reuse malicious samples. As shown in Fig. 10a, SOFA achieves 0.9940 ACC in 8-way 15-shot scenarios and 0.9577 in 8-way 5-shot cases. In contrast, RFS [27] reaches 0.9348 in 8-way 15-shot but drops 12.55% with 5 samples. SOFA outperforms others by 16.44% on average on CICIDS2017 [58] and 14.99% on Botnet2014 [59], owing to its two-stage design. Other methods struggle with sparse benign traffic (easily confused with attacks). KNN-Siamese [38], despite using KNN for boundary samples, fails to capture benign traffic distribution due to sparsity.

We further compared time costs on the CICIDS2017 [58] dataset. As shown in Fig. 11, under the 8-way 10-shot setup (excluding code-dependent feature extraction), our method's training time, which combines VAE training for 31 service triplets (18,718 samples) and the Siamese network, averages 669 seconds (669s) across 10 runs. This exceeds RFS [27] (457s) and PBCNN [37] (537s) but is far more efficient than CNN-MAML [25] (3,049s) and KNN-Siamese [38] (2,916s). For inference on 73,365 samples, our method leads with 4,192 samples/s, outperforming FS-IDS [30] (3,012 samples/s). Module breakdown (Fig. 11b) shows VAE training (162s) is unaffected by N-way K-shot settings, while VAE testing filters 73,365 samples in 1.6s. Pair construction takes about 20s; FSL training varies by shot (155s for 8-way 5-shot, 521s for 8-way 10-shot, 1,043s for 8-way 15-shot), and FSL testing takes about 16s for 34,334 samples. FSL training is the most time-consuming, a common trait of Siamese architectures. Despite partitioning data into multiple service models, VAE training remains efficient, with each model achieving high accuracy in 100 epochs.

D. Binary Classification Anomaly Detection

Then, we evaluate the binary classification anomaly detection methods. Since these methods only distinguish between “benign” and “attack”, we only use the one-class model module in Section V-B for the binary classification experiments. The biggest advantage of our method is that it can group

TABLE III
BINARY CLASSIFICATION ANOMALY DETECTION RESULTS

Model	CICIDS2017				Botnet2014			
	Acc	Pre	Rec	F1	Acc	Pre	Rec	F1
Kitsune	0.9495 0.0112	0.8925 0.0141	0.9784 0.0054	0.9335 0.0088	0.8956 0.0163	0.7330 0.0480	0.8817 0.0133	0.8005 0.0127
FOSS	0.9724 0.0050	0.9669 0.0048	0.9565 0.0056	0.9617 0.0054	0.9062 0.0101	0.8739 0.0104	0.9495 0.0115	0.9101 0.0110
Trident	0.8923 0.0169	0.9350 0.0118	0.8995 0.0150	0.9169 0.0120	0.9115 0.0091	0.8760 0.0163	0.9504 0.0040	0.9117 0.0109
HyperV.	0.9598 0.0043	0.9350 0.0085	0.9401 0.0119	0.9375 0.0099	0.9437 0.0094	0.9376 0.0118	0.9296 0.0118	0.9336 0.0118
Whisper	0.9341 0.0108	0.8922 0.0138	0.9339 0.0084	0.9126 0.0103	0.9295 0.0086	0.9327 0.0094	0.9460 0.0120	0.9403 0.0118
FC-Net	0.9363 0.0107	0.9018 0.0117	0.9201 0.0101	0.9109 0.0112	0.8953 0.0164	0.9067 0.0106	0.8836 0.0124	0.8950 0.0116
SOFA	0.9982 0.0044	1.0000 0.0001	0.9860 0.0055	0.9929 0.0046	0.9716 0.0054	0.9844 0.0041	0.9836 0.0046	0.9840 0.0054

The smaller red fonts denote the standard deviation.

benign traffic in a more fine-grained manner. We build a binary classification model based on the benign traffic in each triplet. The experimental comparison results of binary classification are shown in Table III. The performance table includes the standard deviation (in smaller red font) and the mean. Two attack-related datasets (CICIDS2017 [58] and Botnet2014 [59]) are used for evaluation, and only using their benign traffic to train the baselines and SOFA. The results in the table show that our method has the best recognition effect on both datasets. The autoencoder used in Trident [14] has deeper layers. However, the recognition effect is far inferior to SOFA, which further illustrates the effectiveness of building a model based on service triples.

E. Multi-Classification of Attacks Under Few-Shot Settings

In the previous supervised and binary classification detection comparison experiments, the main reason why the SOFA method performed well is the two-stage processing logic. In the first stage, benign traffic is excluded through binary classification. In the second stage, known attack categories are identified through multi-classification. Benign traffic categories are diverse and are likely to be similar to attack in some dimensions, resulting in an increase in the recognition error rate of SOTA. In order to more deeply demonstrate the superiority of our Siamese network, we exclude benign traffic in this section and only use different types of attack traffic for multi-classification experimental comparison. The entire experiment was conducted on four data sets. Each method was compared in multi-classification detection when the training sample size was 5, 10, and 15 for each class, respectively. The comparative experimental results are shown in Fig. 13. The figures are drawn based on the average and standard deviation of multiple experiments of the overall test ACC of all categories.

As can be seen from Fig. 13, SOFA is the best on all four datasets. On the CICIDS2017 [58] dataset, the malicious behavior features vary greatly, and SOFA can achieve 99.99% ACC. Other methods have also achieved good recognition results, such as ET-BERT [12]. SOFA performs better when the types of malicious attack behaviors exhibit high similarity. For example, on the Botnet2014 [59] dataset, the average recognition accuracy is improved by 10.9% compared to the second-best model, and on the other two datasets, it

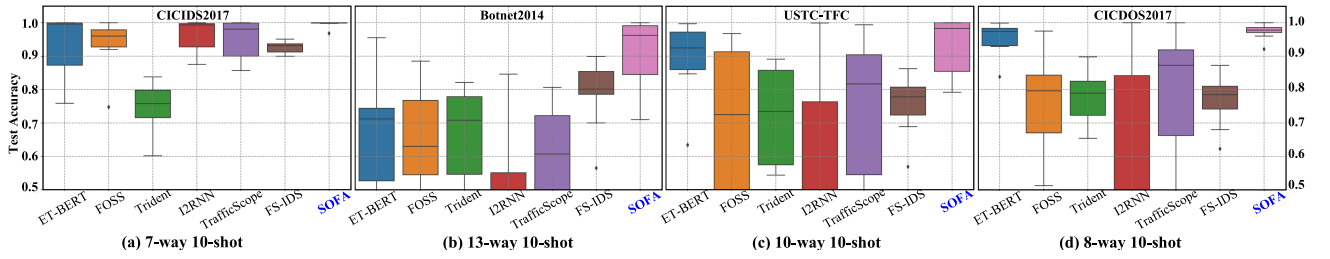


Fig. 12. Comparative analysis of test accuracy on four datasets when using 10 samples per class as the training set.

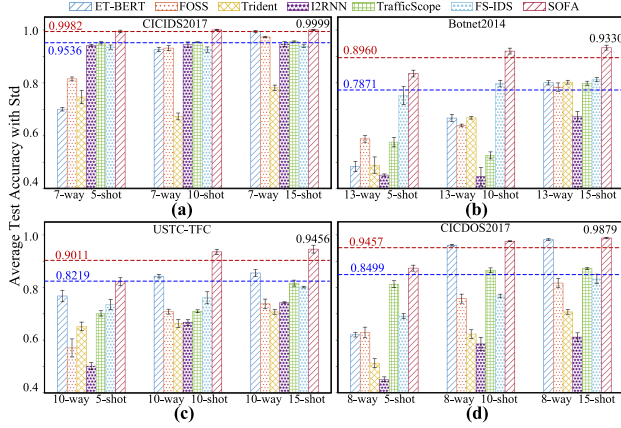


Fig. 13. Comparison of test ACC. The red dashed line —represents the mean accuracy of the optimal model, and the blue dashed line —represents the suboptimal one.

is improved by 7.9% and 9.58% respectively. It is worth noting that our method is more effective when the sample size is smaller. To further explore the recognition accuracy of different models on each category of the dataset, we drew box plots using the test ACC scores from the recognition results on the four datasets, where each box represents the interquartile range (IQR) between the 25th and 75th percentiles, the line inside denotes the median (50th percentile). Fig. 12 shows the recognition accuracy of different models on each category when 10 samples are extracted from each class for training. It can be clearly seen from the figure that although the recognition performance of SOFA has slightly decreased in some categories, the overall recognition effect is still the best.

F. Unknown Detection With Various Proportions

In this section, we evaluate the capability of SOFA by varying $N_k:N_u$ (representing the number of known and unknown attack classes). For CICIDS2017 [58] dataset, we set $N_k:N_u = \{7:0, 6:1, 5:2, 4:3, 3:4, 2:5, 1:6\}$. We set $N_k:N_u = \{13:0, 11:2, 9:4, 7:6, 5:8, 3:10\}$ for Botnet2014 [59] dataset. For dataset USTC-TFC [62], we set the known and unknown ratios $N_k:N_u$ to $\{10:0, 8:2, 6:4, 4:6, 2:8\}$, while for the CICDOS2017 [63] dataset, we set them as $\{8:0, 7:1, 6:2, 4:4, 2:6\}$. The experiment sets the number of samples in each category used for meta-training to 15. When experimenting on each dataset, the categories used for meta-training were randomly selected. For example, when $N_k:N_u = 6:1$, We randomly selected 6 categories out of 7 with 15 samples per category for meta-training and used the remaining data from all 7 categories for testing. 10 experiments were conducted under

each setting, and the categories used for meta-training were randomly re-selected each time to ensure sufficient robustness. The comparative experimental results are shown in Fig. 14. As can be seen from the figure, SOFA has the best ACC for unknown category recognition under any ratio condition. We marked the average ACC above the bar. As can be seen from subfigure (a), SOFA's test ACC is not affected by the ratio at all on the CICIDS2017 [58] dataset, which actually shows that each attack behavior on the CICIDS2017 [58] dataset has obvious features. On other datasets, as the ratio of known and unknown categories increases, the test ACC decreases slightly. This is determined by the similarity of attack category samples. When similar attack types appear in unknown categories, it is easy to cause an increase in the recognition error rate. The ACC is highly related to the threshold set for each category. We will discuss in depth how to set the threshold to enhance the ACC in Section VII-B.

G. Ablation Study

Threshold Ablation. In Section V-B, we proposed two anomaly detection methods based on the VAE one-class model. In Equation (16), the preset threshold method requires the quantile α to be set in advance. In Equation (19), the EVT theory requires the initial threshold t and risk parameter q to be set in advance. In this section, we conduct an ablation study on the sensitivity of these parameters respectively. The experimental results are shown in Fig. 15 and 16. In Fig. 15, the horizontal axis represents the preset t value $t \in [0.90, 0.99]$ (here t represents α in Equation (16)), and the vertical axis represents the average ACC of all categories. The red line with standard deviation in the figure represents the impact of EVT on test ACC under different t values. The green line represents the preset threshold method. From the results on the two datasets, we can see that the EVT method has good robustness to t . As t becomes smaller, the ACC decreases slightly. On the CICIDS2017 [58] dataset, when t changes from 0.99 to 0.90, the ACC decreases by about 0.0294; on the Botnet2014 [59] dataset, the ACC decreases by about 0.0163. Compared with the preset threshold method (green line), EVT is less sensitive to t . In order to test the sensitivity of the EVT method to the two parameters t and q , we conducted accuracy tests under different t ($t \in [0.90, 0.99]$) and q ($q \in [0.01, 0.09]$), as shown in Fig. 16. When t is fixed (for example, $t = 0.95$), different q has little effect on the ACC, and the maximum ACC decreases by 0.0021. In general, the EVT method has certain robustness to the two parameters t and q , and is more stable than the preset threshold method.

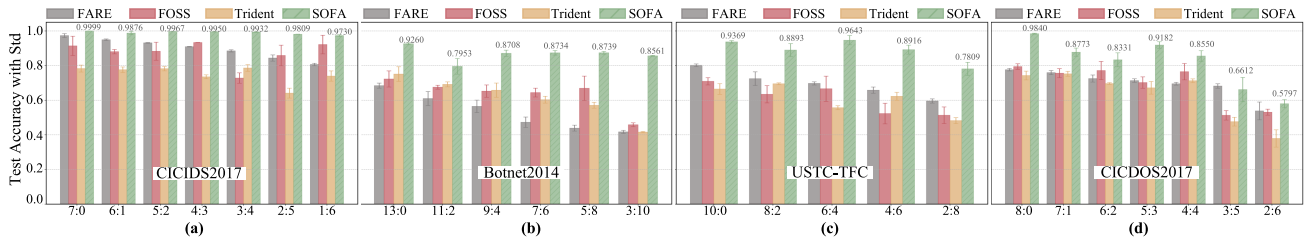


Fig. 14. Evaluation at varying $N_k:N_u$ ratios. Shown is the test ACC after training on 15 samples of each category.

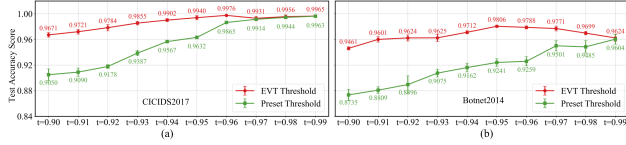


Fig. 15. Sensitivity of threshold t for EVT and preset method.

$q=0.09$	0.9421 ±0.0043	0.9644 ±0.0015	0.9615 ±0.0287	0.9578 ±0.0062	0.9799 ±0.0039	0.9679 ±0.0087	0.9725 ±0.0085	0.9754 ±0.0111	0.9710 ±0.0098	0.9599 ±0.0021
$q=0.07$	0.9432 ±0.0019	0.9641 ±0.0012	0.9678 ±0.0085	0.9639 ±0.0070	0.9801 ±0.0057	0.9769 ±0.0115	0.9759 ±0.0089	0.9769 ±0.0106	0.9733 ±0.0018	0.9644 ±0.0018
$q=0.05$	0.9449 ±0.0015	0.9651 ±0.0012	0.9689 ±0.0096	0.9657 ±0.0070	0.9803 ±0.0057	0.9782 ±0.0116	0.9759 ±0.0106	0.9775 ±0.0112	0.9752 ±0.0094	0.9654 ±0.0018
$q=0.03$	0.9423 ±0.0021	0.9635 ±0.0014	0.9682 ±0.0096	0.9643 ±0.0101	0.9790 ±0.0058	0.9782 ±0.0116	0.9733 ±0.0106	0.9765 ±0.0112	0.9715 ±0.0099	0.9654 ±0.0018
$q=0.01$	0.9445 ±0.0017	0.9651 ±0.0015	0.9633 ±0.0108	0.9641 ±0.0070	0.9782 ±0.0058	0.9751 ±0.0116	0.9710 ±0.0106	0.9731 ±0.0112	0.9709 ±0.0094	0.9642 ±0.0018
	$t=0.90$	$t=0.91$	$t=0.92$	$t=0.93$	$t=0.94$	$t=0.95$	$t=0.96$	$t=0.97$	$t=0.98$	$t=0.99$

Fig. 16. Sensitivity of risk parameter q and threshold t .

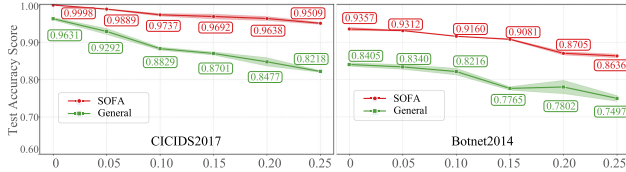


Fig. 17. Ablation study on feature enhancement.

Feature Enhancement Ablation. In Section V-C, in order to reduce the impact of noise on few-shot learning, we adopted a feature enhancement method, that is, mapping the features of the current sample and its neighbors (with the same client IP and from the same service triplet) to itself. This approach was also used in the testing process. To illustrate the effectiveness of this feature enhancement method, we conducted an ablation experiment. On the CICIDS2017 [58] and Botnet2014 [59] datasets, we used feature enhancement technology and general methods to test the impact of the Siamese network on fine-grained malicious behavior detection. We refer to the obfuscation evasion attack proposed in [6] and [22] and simulate attackers injecting benign traffic into various categories of malicious attack traffic at different ratios $p \in [0, 0.25]$. Ten experiments with different obfuscation ratios were conducted on two datasets to test the noise suppression capability of feature enhancement methods. Specifically, for a malicious category c_i with n_i samples, we locate the service triplet targeted by attack traffic, randomly sample $k_i = n_i \times pp = 0, 0.05, \dots, 0.25$ benign samples from this triplet, and mix them into the malicious category before model training and testing. As shown in Fig. 17, the horizontal axis represents the ratios of injected benign samples while the vertical axis shows the overall average ACC of the Siamese network detection. On the CICIDS2017 [58] dataset, the SOFA method maintains remarkable stability as the obfuscation ratio

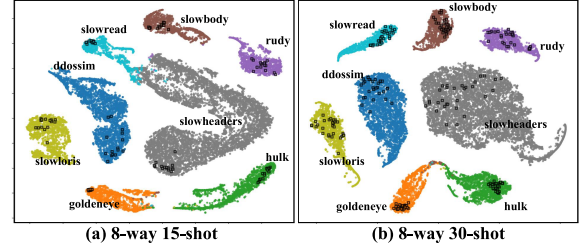


Fig. 18. Evaluation results of Siamese network on CICDOS2017 dataset.

increases with ACC dropping by only 4.89% when p rises from 0 to 0.25 compared to a 14.13% decrease in the General method. On the Botnet2014 [59] dataset, due to inherent noise in dataset, SOFA achieves an average ACC of 9.52% higher than the General method without obfuscation and retains an identification ACC of 0.8636 at $p = 0.25$ versus 0.7497 for the General method. Overall, the feature enhancement method effectively suppresses obfuscation attacks with an average 9.62% improvement in detection accuracy across both datasets.

VII. DISCUSSION AND LIMITATIONS

A. Deep Insights Into Feature Space

The Siamese network effectively learns a metric space, reducing intra-class sample distances while expanding inter-class ones. Visualizations on the CICDOS2017 [63] dataset illustrate this. Fig. 18a and Fig. 18b show model-fitted results using 15 and 30 training samples respectively via t-SNE, with black dots representing training samples and colored dots denoting test samples. Small sample distribution critically impacts accuracy. When the samples used for training are representative (e.g., *Rudy* attack), the accuracy is close to 100% with 15 samples. However, overly concentrated distributions limit decision boundaries, causing misclassification for test samples with concept drift (e.g., *Slowheaders* in Fig. 18a). Increasing samples to 30 improves recognition of previously poorly performing classes like *Slowheaders* and *Slowreads* (see Fig. 18b), as more samples better cover the data distribution range.

B. Deep Insights in Unknown Attack Identification

This section highlights the effectiveness of the adaptive thresholding method with a global lower bound (Equation (30)) and its limitations. The global lower bound τ'_c must lie within each class's threshold interval $I_{c_i} = [\tau_{c_i}^{min}, \tau_{c_i}^{max}]$, where $\tau_{c_i}^{min} = \mu_{c_i}$ and the maximum threshold is calculated as:

$$\tau_{c_i}^{max} = \frac{1}{2} \min_{j \neq i} d(\mu_{c_i}, \mu_{c_j}) \quad (31)$$

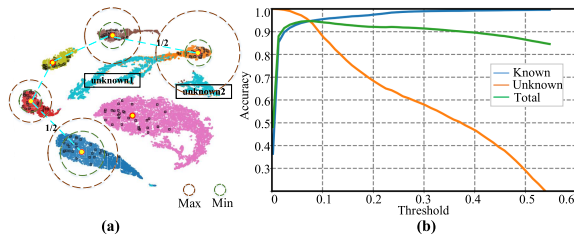


Fig. 19. Relationship between threshold and test ACC.

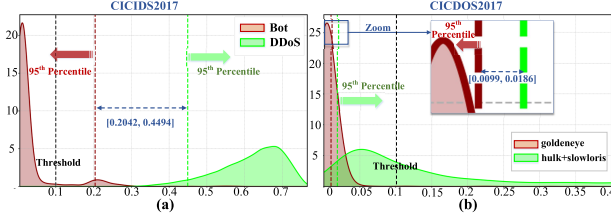


Fig. 20. Probability density distribution of distances from test samples to the center of known classes, where the green region denotes unknown classes and the red one represents known classes.

We empirically set $\tau'_c = 0.1$, which balances accuracy for both known and unknown categories on the CICDOS2017 [63] dataset. As shown in Fig. 19, lower thresholds favor unknown category detection, while higher ones benefit known categories, but 0.1 achieves optimal overall performance. Experiments on two datasets further validate this. For CICIDS2017 [58] (with *DDoS* as unknown), the threshold calculated in Equation (30) achieves over 95% accuracy, as distinct malicious features allow robust detection within [0.1, 0.5]. For CICDOS2017 [63] (with *Hulk* and *Slowloris* as unknown), overlapping features with *Goldeneyes* limit accuracy to about 85%, even with optimized thresholds, indicating the need for supplementary strategies (e.g., temporal patterns, semi-supervised learning) in such cases. In summary, the combined local-global thresholding works well for small samples but requires more samples to optimize thresholds for complex datasets with overlapping attack features.

C. Model Deployment and Update

Two issues need to be addressed when deploying and using the model: (i) Handling unknown service triplets. SOFA incorporates a regular automatic update mechanism: the service aggregation module runs periodically to identify service triplets requiring updates. For instance, when an enterprise launches a new external service, a large volume of related traffic is generated. The module then verifies the service's legitimacy. If confirmed as a normal internal service triplet, its traffic is collected to train and update the VAE one-class model, leaving other models unchanged. Any external attempts to access the enterprise's illegal service triplets are recorded as attacks. (ii) Updating existing service triplets. SOFA employs a *back-to-front* reverse strategy: if continuous unknown attack behaviors are detected in the Siamese network phase, the system attributes this to potential errors in the VAE detection phase. It conducts retrospective analysis on these unknown traffic categories to identify which service triplet has undergone business updates, then recollects traffic for model update training.

Furthermore, it is worth emphasizing that the real-time performance optimization in practical deployment [71], [72] is orthogonal to the core design of our SOFA framework, which allows for flexible integration with high-performance data-plane technologies (e.g., combining with Intel DPDK [6], [22], [68]) to enhance throughput and reduce latency in high-traffic scenarios. The service-oriented traffic segregation in its first stage inherently supports parallel processing, and the inference logic of the second-stage Siamese network can be converted into lightweight decision rules [73]. These deployment optimizations can be tailored to specific hardware environments to meet line-speed requirements while preserving the algorithm's advantages in detection accuracy.

D. Limitations and Future Work

Our work has a few limitations. First, our method did not adopt a parallel strategy when training one-class models in the first stage. If the enterprise network is large enough and has many open services, model training and testing can be run on multiple machines in parallel. Second, although threshold-based anomaly detection is flexible in practice, it is not accurate enough. In the case of insufficient sample size, an adaptive threshold calculation method based on unlabeled data can be considered [74].

VIII. CONCLUSION

In this paper, we present SOFA, a novel two-stage framework designed to address the challenges of fine-grained unknown attack detection under the constraints of class imbalance and limited labeled attack samples. By introducing service triplet-based aggregation and combining one-class learning with meta-learning principles, SOFA achieves precise detection of both known and unknown attacks. Key contributions include the identification of service triplets through graph-based aggregation, the design of Variational AutoEncoders for service-specific anomaly detection, and the utilization of a ResNet-based Siamese network for fine-grained classification and unknown attack identification. Through extensive experiments on four widely-used network intrusion detection datasets, SOFA demonstrated state-of-the-art performance, significantly outperforming a diverse set of baselines across both binary and multi-class classification tasks. Notably, the framework excelled in handling dynamic, highly imbalanced threat environments and showed effectiveness in detecting emerging and unknown attack behaviors.

REFERENCES

- [1] B. Nour, M. Pourzandi, and M. Debbabi, "A survey on threat hunting in enterprise networks," *IEEE Commun. Surveys Tut.*, vol. 25, no. 4, pp. 2299–2324, 4th Quart., 2023.
- [2] Z. Zhao et al., "CMD: Co-analyzed IoT malware detection and forensics via network and hardware domains," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5589–5603, May 2024.
- [3] Z. Zhao et al., "ERNN: Error-resilient RNN for encrypted traffic detection towards network-induced phenomena," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 3, 2023, doi: [10.1109/TDSC.2023.3242134](https://doi.org/10.1109/TDSC.2023.3242134).
- [4] A. Saha, N. Ganguly, S. Chakraborty, and A. De, "Learning network traffic dynamics using temporal point process," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Apr. 2019, pp. 1927–1935.

- [5] F. Xu, Y. Li, H. Wang, P. Zhang, and D. Jin, "Understanding mobile traffic patterns of large scale cellular towers in urban environment," *IEEE/ACM Trans. Netw.*, vol. 25, no. 2, pp. 1147–1161, Apr. 2017.
- [6] C. Fu, Q. Li, M. Shen, and K. Xu, "Realtime robust malicious traffic detection via frequency domain analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3431–3446.
- [7] H. Li, H. Hu, G. Gu, G.-J. Ahn, and F. Zhang, "VNIDS: Towards elastic security with safe and efficient virtualization of network intrusion detection systems," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS)*, Oct. 2018, pp. 17–34.
- [8] Z. Zhao et al., "FOSS: Towards fine-grained unknown class detection against the open-set attack spectrum with variable legitimate traffic," *IEEE/ACM Trans. Netw.*, vol. 32, no. 5, pp. 3945–3960, Oct. 2024.
- [9] P. Jiang et al., "Building IN-THE-CLOUD network functions: Security and privacy challenges," *Proc. IEEE*, vol. 109, no. 12, pp. 1888–1919, Dec. 2021.
- [10] Z. Song et al., "I² RNN: An incremental and interpretable recurrent neural network for encrypted traffic classification," *IEEE Trans. Dependable Secure Comput.*, early access, Feb. 28, 2023, doi: [10.1109/TDSC.2023.3245411](https://doi.org/10.1109/TDSC.2023.3245411).
- [11] T. Shapira and Y. Shavitt, "FlowPic: A generic representation for encrypted traffic classification and applications identification," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 2, pp. 1218–1232, Jun. 2021.
- [12] X. Lin, G. Xiong, and G. Gou, "ET-BERT: A contextualized data-gram representation with pre-training transformers for encrypted traffic classification," in *Proc. ACM Web Conf.*, 2022, pp. 633–642.
- [13] X. Shu, D. Yao, and N. Ramakrishnan, "Unearthing stealthy program attacks buried in extremely long execution paths," in *Proc. 22nd ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2015, pp. 401–413.
- [14] Z. Zhao, Z. Li, Z. Song, W. Li, and F. Zhang, "Trident: A universal framework for fine-grained and class-incremental unknown traffic detection," in *Proc. ACM Web Conf.*, May 2024, pp. 1608–1619.
- [15] J. Yang, X. Chen, S. Chen, X. Jiang, and X. Tan, "Conditional variational auto-encoder and extreme value theory aided two-stage learning approach for intelligent fine-grained known/unknown intrusion detection," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 3538–3553, 2021.
- [16] A. Thakkar and R. Lohiya, "Attack classification of imbalanced intrusion data for IoT network using ensemble-learning-based deep neural network," *IEEE Internet Things J.*, vol. 10, no. 13, pp. 11888–11895, Jul. 2023.
- [17] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.
- [18] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Apr. 2018.
- [19] J. Liang, W. Guo, T. Luo, V. Honavar, G. Wang, and X. Xing, "FARE: Enabling fine-grained attack categorization under low-quality labeled data," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–18.
- [20] C. Sheng, Y. Yao, W. Li, W. Yang, and Y. Liu, "Unknown attack traffic classification in SCADA network using heuristic clustering technique," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2625–2638, Sep. 2023.
- [21] G. Münz, S. Li, and G. Carle, "Traffic anomaly detection using k-means clustering," in *Proc. GI/ITG Workshop MMBNet*, 2007, vol. 7, no. 9, pp. 1–8.
- [22] C. Fu, Q. Li, and K. Xu, "Detecting unknown encrypted malicious traffic in real time via flow interaction graph analysis," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [23] M. Kim, W. Jang, J. Hur, and M. Yoon, "Relative frequency-rank encoding for unsupervised network anomaly detection," *IEEE/ACM Trans. Netw.*, vol. 32, no. 4, pp. 3453–3467, Aug. 2024.
- [24] M. T. R. Laskar et al., "Extending isolation forest for anomaly detection in big data via K-means," *ACM Trans. Cyber-Phys. Syst.*, vol. 5, no. 4, pp. 1–26, Oct. 2021.
- [25] C. Lu, X. Wang, A. Yang, Y. Liu, and Z. Dong, "A few-shot-based model-agnostic meta-learning for intrusion detection in security of Internet of Things," *IEEE Internet Things J.*, vol. 10, no. 24, pp. 21309–21321, Dec. 2023.
- [26] T. Wang, Q. Lv, B. Hu, and D. Sun, "A few-shot class-incremental learning approach for intrusion detection," in *Proc. Int. Conf. Comput. Commun. Netw. (ICCCN)*, Jul. 2021, pp. 1–8.
- [27] D. Di Monda, A. Montieri, V. Persico, P. Voria, M. De Ieso, and A. Pescapè, "Few-shot class-incremental learning for network intrusion detection systems," *IEEE Open J. Commun. Soc.*, vol. 5, pp. 6736–6757, 2024.
- [28] L. Du, Z. Gu, Y. Wang, L. Wang, and Y. Jia, "A few-shot class-incremental learning method for network intrusion detection," *IEEE Trans. Netw. Service Manage.*, vol. 21, no. 2, pp. 2389–2401, Apr. 2024.
- [29] Y. Chai, L. Du, J. Qiu, L. Yin, and Z. Tian, "Dynamic prototype network based on sample adaptation for few-shot malware detection," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 5, pp. 4754–4766, May 2023.
- [30] J. Yang, H. Li, S. Shao, F. Zou, and Y. Wu, "FS-IDS: A framework for intrusion detection based on few-shot learning," *Comput. Secur.*, vol. 122, Nov. 2022, Art. no. 102899.
- [31] S. Huang et al., "A gated few-shot learning model for anomaly detection," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Jan. 2020, pp. 505–509.
- [32] D. D. Monda, G. Bovenzi, A. Montieri, V. Persico, and A. Pescapè, "IoT botnet-traffic classification using few-shot learning," in *Proc. IEEE Int. Conf. Big Data (BigData)*, Dec. 2023, pp. 3284–3293.
- [33] W. Zheng, C. Gou, L. Yan, and S. Mo, "Learning to classify: A flow-based relation network for encrypted traffic classification," in *Proc. Web Conf.*, Apr. 2020, pp. 13–22.
- [34] C. Xu, J. Shen, and X. Du, "A method of few-shot network intrusion detection based on meta-learning framework," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 3540–3552, 2020.
- [35] W. Liang, Y. Hu, X. Zhou, Y. Pan, and K. I. Wang, "Variational few-shot learning for microservice-oriented intrusion detection in distributed industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 8, pp. 5087–5095, Aug. 2022.
- [36] Y. Chen, Z. Li, J. Shi, G. Gou, C. Liu, and G. Xiong, "Not afraid of the unseen: A Siamese network based scheme for unknown traffic discovery," in *Proc. IEEE Symp. Comput. Commun. (ISCC)*, Jul. 2020, pp. 1–7.
- [37] L. Yu et al., "PBCNN: Packet bytes-based convolutional neural network for network intrusion detection," *Comput. Netw.*, vol. 194, Jul. 2021, Art. no. 108117.
- [38] M. Pawlicki, R. Kozik, and M. Choraś, "Improving Siamese neural networks with border extraction sampling for the use in real-time network intrusion detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jun. 2023, pp. 1–8.
- [39] C. Finn, P. Abbeel, and S. Levine, "Model-agnostic meta-learning for fast adaptation of deep networks," in *Proc. 34th Int. Conf. Mach. Learn.*, vol. 70, Aug. 2017, pp. 1126–1135.
- [40] Y. Tian, Y. Wang, D. Krishnan, J. B. Tenenbaum, and P. Isola, "Rethinking few-shot image classification: A good embedding is all you need?," in *Proc. ECCV*, 2020, pp. 266–282.
- [41] J. Wang and Y. Zhai, "Prototypical Siamese networks for few-shot learning," in *Proc. IEEE 10th Int. Conf. Electron. Inf. Emergency Commun. (ICEIEC)*, Jul. 2020, pp. 178–181.
- [42] O. Vinyals, C. Blundell, T. Lillicrap, K. Kavukcuoglu, and D. Wierstra, "Matching networks for one shot learning," in *Proc. NeurIPS*, vol. 29, 2016, pp. 3637–3645.
- [43] F. Sung, Y. Yang, L. Zhang, T. Xiang, P. H. S. Torr, and T. M. Hospedales, "Learning to compare: Relation network for few-shot learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 1199–1208.
- [44] L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi, and P. H. S. Torr, "Fully-convolutional Siamese networks for object tracking," in *Proc. ECCV Workshops*. Cham, Switzerland: Springer, 2016, pp. 850–865.
- [45] G. Bovenzi, D. Di Monda, A. Montieri, V. Persico, and A. Pescapè, "Classifying attack traffic in IoT environments via few-shot learning," *J. Inf. Secur. Appl.*, vol. 83, Jun. 2024, Art. no. 103762.
- [46] H. Hindy, C. Tachtatzis, R. Atkinson, E. Bayne, and X. Bellekens, "Developing a Siamese network for intrusion detection systems," in *Proc. 1st Workshop Mach. Learn. Syst. (EuroMLSys)*, 2021, pp. 120–126.
- [47] K. Thomas et al., "Data breaches, phishing, or malware?: Understanding the risks of stolen credentials," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1421–1434.
- [48] T. Jirsik and P. Velan, "Host behavior in computer network: One-year study," *IEEE Trans. Netw. Service Manage.*, vol. 18, no. 1, pp. 822–838, Mar. 2021.
- [49] Y. Mirsky, T. Doitshman, Y. Elovici, and A. Shabtai, "Kitsune: An ensemble of autoencoders for online network intrusion detection," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [50] J. Beirlant, Y. Goegebeur, J. Segers, and J. L. Teugels, *Statistics of Extremes: Theory and Applications*. Hoboken, NJ, USA: Wiley, 2006.

- [51] A. Siffer, P.-A. Fouque, A. Termier, and C. Largouet, "Anomaly detection in streams with extreme value theory," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Aug. 2017, pp. 1067–1075.
- [52] J. Pickands III, "Statistical inference using extreme order statistics," *Ann. Statist.*, vol. 3, no. 1, pp. 119–131, Jan. 1975.
- [53] D. Meng and H. Chen, "MagNet: A two-pronged defense against adversarial examples," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 135–147.
- [54] W. Xu, D. Evans, and Y. Qi, "Feature squeezing: Detecting adversarial examples in deep neural networks," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [55] Y. Yang, R. Gao, Y. Li, Q. Lai, and Q. Xu, "What you see is not what the network infers: Detecting adversarial examples based on semantic contradiction," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022, pp. 1–20.
- [56] Z. Zhao, Z. Li, T. Li, Z. Song, F. Zhang, and R. Zhang, "Poster: Detecting adversarial examples hidden under watermark perturbation via usable information theory," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2023, pp. 3636–3638.
- [57] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. USENIX Secur. Symp.*, 2022, pp. 3971–3988.
- [58] Canadian Institute for Cybersecurity. (2018). *Intrusion Detection Evaluation Dataset (CICIDS2017)*. Canadian Institute for Cybersecurity. Accessed: Nov. 27, 2024. [Online]. Available: <https://www.unb.ca/cic/datasets/ids-2017.html>
- [59] E. Biglar Beigi, H. Hadian Jazi, N. Stakhanova, and A. A. Ghorbani, "Towards effective feature selection in machine learning-based botnet detection approaches," in *Proc. IEEE Conf. Commun. Netw. Secur.*, Oct. 2014, pp. 247–255.
- [60] E. W. Weisstein. *Cumulative Distribution Function*. Accessed: Jul. 12, 2024. [Online]. Available: <https://mathworld.wolfram.com/CumulativeDistributionFunction.html>
- [61] T. J. Ypma, "Historical development of the Newton–Raphson method," *SIAM Rev.*, vol. 37, no. 4, pp. 531–551, Dec. 1995.
- [62] W. Wang, M. Zhu, X. Zeng, X. Ye, and Y. Sheng, "Malware traffic classification using convolutional neural network for representation learning," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, Jan. 2017, pp. 712–717.
- [63] H. H. Jazi, H. Gonzalez, N. Stakhanova, and A. A. Ghorbani, "Detecting HTTP-based application layer DoS attacks on web servers in the presence of sampling," *Comput. Netw.*, vol. 121, pp. 25–36, Jul. 2017.
- [64] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Commun. Netw. Secur. (CNS)*, Oct. 2022, pp. 254–262.
- [65] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, May 2021, pp. 7–12.
- [66] A. Rosay, F. Carlier, E. Cheval, and P. Leroux, "From CIC-IDS2017 to LYCOS-IDS2017: A corrected dataset for better performance," in *Proc. IEEE/WIC/ACM Int. Conf. Web Intell.*, Dec. 2021, pp. 570–575.
- [67] T. van Ede et al., "FlowPrint: Semi-supervised mobile-app fingerprinting on encrypted network traffic," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–18.
- [68] Z. Zhao et al., "Towards context-aware traffic classification via time-wavelet fusion network," in *Proc. 31st ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Jul. 2025, pp. 2089–2100.
- [69] J. Holland, P. Schmitt, N. Feamster, and P. Mittal, "New directions in automated traffic analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 3366–3383.
- [70] R. Tang et al., "ZeroWall: Detecting zero-day web attacks through encoder-decoder recurrent neural networks," in *Proc. IEEE INFOCOM - IEEE Conf. Comput. Commun.*, Jul. 2020, pp. 2479–2488.
- [71] Z. Zhao, Z. Li, Z. Song, F. Zhang, and B. Chen, "RIDS: Towards advanced IDS via RNN model and programmable switches co-designed approaches," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, May 2024, pp. 591–600.
- [72] Z. Zhao et al., "Verify all traffic: Towards zero-trust in-network intrusion detection against multipath routing," *IEEE J. Sel. Areas Commun.*, vol. 43, no. 6, pp. 2155–2171, Jun. 2025.
- [73] Z. Zhao, Z. Liu, H. Chen, F. Zhang, Z. Song, and Z. Li, "Effective DDoS mitigation via ML-driven in-network traffic shaping," *IEEE Trans. Dependable Secure Comput.*, vol. 21, no. 4, pp. 4271–4289, Jul. 2024.
- [74] Y. Wang et al., "FreeMatch: Self-adaptive thresholding for semi-supervised learning," 2022, *arXiv:2205.07246*.



Peng Fang received the M.S. degree from the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China. He is currently pursuing the Ph.D. degree with Zhejiang University, Hangzhou, China. His main research interests include encrypted traffic analysis, threat detection, and deep learning.



Feiyang Huang received the B.S. degree from the School of Cyber Science and Engineering, Sichuan University, Chengdu, China. He is currently a Graduate Student with Zhejiang University, Hangzhou, China. His main research interests include encrypted traffic analysis and deep learning.



Ziming Zhao (Member, IEEE) is a ZJU 100 Young Professor with the School of Software Technology, Zhejiang University, China. He has published more than 40 papers in international journals and conference proceedings, including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, MOBI-COM, SIGCOMM, WWW, INFOCOM, KDD, CCS, RTSS, IJCAI, IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS, IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON INFORMATION FORENSICS AND SECURITY, IEEE TRANSACTIONS ON DEPENDABLE AND SECURE COMPUTING, COSE, DATE, ESE, and AAAI. His research interests include machine learning, traffic identification, network security, and mobile computing.



Fan Zhang (Member, IEEE) received the Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, CT, USA, in 2011. He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China. His research interests include system security, hardware security, network security, cryptography, and computer architecture.



Wenrui Ma was born in 1985. She received the Ph.D. degree in computer science from Florida International University in 2018. She is currently an Assistant Professor with the School of Computer Science and Technology, Zhejiang Gongshang University. Her research interests include high performance network architecture and network security.